

ICS Exercise — Week 7

April 16, 2026

Problem 1: Signal Handler Concurrency

Read the program below carefully. `Signal`, `Sigprocmask`, `Fork`, `Kill`, `Pause`, `Sigemptyset`, and `Sigaddset` are the CSAPP wrappers (they exit on error). `sio_puts` and `sio_putl` are the async-signal-safe I/O helpers from the lecture.

```
1  #include "csapp.h"
2  volatile sig_atomic_t reaped_n = 0;
3  volatile sig_atomic_t usrl_n   = 0;
4
5  void handler_chld(int sig) {
6      int saved_errno = errno;
7      pid_t pid;
8      while ((pid = waitpid(-1, NULL, WNOHANG)) > 0) {
9          sio_puts("reaped\n");
10         reaped_n++;
11     }
12     errno = saved_errno;
13 }
14
15 void handler_usr1(int sig) {
16     int saved_errno = errno;
17     sio_puts("u1\n");
18     usrl_n++;
19     errno = saved_errno;
20 }
21
22 int main(void) {
23     sigset_t mask_chld, prev;
24     Sigemptyset(&mask_chld);
25     Sigaddset(&mask_chld, SIGCHLD);
26
27     Signal(SIGCHLD, handler_chld);
28     Signal(SIGUSR1, handler_usr1);
29
30     Sigprocmask(SIG_BLOCK, &mask_chld, &prev);
31
32     for (int i = 0; i < 3; i++) {
33         if (Fork() == 0) {
34             Kill(getppid(), SIGUSR1);
35             exit(0);
36         }
37     }
38
39     sio_puts("loop done\n");
40     Sigprocmask(SIG_SETMASK, &prev, NULL);
41
42     while (reaped_n < 3) Pause();
43     printf("usrl_n=%d\n", usrl_n);
44     return 0;
45 }
```

1.1 Visibility table

For each line of output below, write **T** if the line is always printed in every legal execution, **F** if it is never printed, or **M** if it might or might not be printed.

Output line	Visibility
L09 <code>reaped\n</code>	[a]
L17 <code>u1\n</code>	[b]
L39 <code>loop done\n</code>	[c]
L43 <code>usr1_n=...\n</code>	[d]

1.2 Possible values

- a. Across every legal execution of the program, what are the possible values of `reaped_n` printed if we logged it after the loop on line 42? Justify in one sentence.
- b. What are the possible values of `usr1_n` printed by line 43? Justify with reference to one specific concept from the signals lecture.

1.3 Race condition

There is a race in the `while (reaped_n < 3) Pause();` loop on line 42. Describe a concrete interleaving in which the program **hangs forever**, and rewrite lines 30 + 40 + 42 using `sigsuspend` so that the hang is impossible.

1.4 Async-signal safety

`printf` is called on line 43 inside `main`, and `sio_puts` is called inside both handlers. Suppose a colleague proposes replacing `sio_puts("reaped\n")` on line 9 with `printf("reaped\n")`. Why is this proposal unsafe? Name the specific guideline (G0/G1/G2) it violates and describe one observable consequence.

Problem 2: Safe Recovery with `sigsetjmp` / `siglongjmp`

Consider this program, which uses a SIGINT handler to implement an automatic restart loop:

```
1  #include "csapp.h"
2  #include <setjmp.h>
3
4  sigjmp_buf env;
5  volatile sig_atomic_t restarts = 0;
6
7  void handler(int sig) {
8      sio_puts("handler\n");
9      restarts++;
10     siglongjmp(env, restarts);
11 }
12
13 int main(void) {
14     int rc;
15     Signal(SIGINT, handler);
16
17     rc = sigsetjmp(env, 1);
```

```

18     printf("sigsetjmp returned %d\n", rc);
19
20     if (rc == 0) {
21         printf("starting work\n");
22     } else if (rc < 3) {
23         printf("restart #%d\n", rc);
24     } else {
25         printf("giving up after %d restarts\n", rc);
26         exit(0);
27     }
28
29     printf("about to raise SIGINT\n");
30     Kill(getpid(), SIGINT);
31     printf("after SIGINT (never reached)\n");
32     return 0;
33 }

```

2.1 Output trace

Write the **complete** output of this program, in order. (Assume `printf` lines are flushed normally.)

2.2 Unreachable code

Why is the line `printf("after SIGINT (never reached)\n")` on line 31 never printed?

2.3 `setjmp` vs. `sigsetjmp`

Suppose we replace `sigsetjmp(env, 1)` with `setjmp(env)` on line 17 and `siglongjmp(env, restarts)` with `longjmp(env, restarts)` on line 10 (and use `jmp_buf` instead of `sigjmp_buf` on line 4). The first iteration would still print correctly. Explain what subtly goes wrong on the **second** iteration. (Hint: think about the signal mask on entry to and exit from the handler.)

Problem 3: Scheduling Policy Comparison

Consider the workload below. Time 0 means the job runs during the interval [0 ms, 1 ms).

Job	Arrival (ms)	Run-time (ms)
A	0	8
B	2	2
C	4	1
D	6	4

Assume:

- **FIFO** is non-preemptive.
- **SJF** is non-preemptive (decisions made only at completion time).
- **STCF** is preemptive (re-evaluate on every arrival).
- **RR** uses a 2 ms quantum. When a quantum ends, any newly-arrived job is appended to the tail of the ready queue **before** the just-preempted job is appended.
- No I/O or context-switch cost.

3.1 Metrics table

Fill in the table below. Use exact fractions or decimals (e.g., 0.75).

Policy	Avg. turnaround	Avg. response	CPU at t = 6 ms	CPU at t = 12 ms
FIFO	[a1]	[a2]	[a3]	[a4]
SJF	[b1]	[b2]	[b3]	[b4]
STCF	[c1]	[c2]	[c3]	[c4]
RR	[d1]	[d2]	[d3]	[d4]

3.2 Best response time

Which policy achieves the smallest average response time on this workload? Explain in 2–3 sentences why it does so well **and** identify one workload property that would make it perform worse than RR.

3.3 Convoy effect

Briefly describe the convoy effect using FIFO on this specific workload, citing the response-time numbers from the table.

Problem 4: MLFQ Trace

Consider the workload below, scheduled by **MLFQ** with two priority queues:

- **High priority** queue: 1 ms time-slice.
- **Low priority** queue: 2 ms time-slice.
- RR scheduling within each queue.
- A higher-priority arrival immediately preempts a running lower-priority job. The preempted job re-enters the tail of its queue and receives a **fresh** time-slice when it next runs.
- A job that uses up its full time-slice is demoted to the lower queue (jobs already on the low queue stay on the low queue and rotate via RR).
- **Priority boost is disabled.**
- New arrivals enter the high-priority queue.

Job	Arrival (ms)	Run-time (ms)
A	0	4
B	2	2
C	4	3

4.1 Fill the schedule

Complete the table below by writing the job that runs in each 1 ms interval.

Time (ms)	0	1	2	3	4	5	6	7	8
CPU	A	[a]	[b]	[c]	[d]	[e]	[f]	[g]	[h]

4.2 Compute metrics

Using your filled-in schedule, compute:

- a. The completion time, turnaround time, and response time for each of A, B, C.
- b. The average turnaround time and average response time across the three jobs.

4.3 Starvation without priority boost

In 2–3 sentences, describe a workload (job arrivals + run-times) in which disabling priority boost causes one job to starve indefinitely under this 2-queue MLFQ. Be specific about which job starves and why.